

오목에 적용하는
위협공간탐색(Threat-Space
Search)

강연자 학번 : 11004

강연자 성명 : 김동건

목 차

I. 서론

1. 탐구의 필요성
2. 탐구의 목적

II. 탐구 내용

1. 위협공간탐색의 개념과 원리
 - 가) 용어의 정의
 - 1) 오목의 용어 정의
 - 2) 위협공간탐색의 용어 정의
 - 나) 위협공간탐색의 원리와 예시
 - 1) 위협공간탐색의 원리
 - 2) 위협공간탐색의 예시
2. 위협공간탐색의 한계와 보완

III. 결론

1. 요약
2. 마무리

I. 서론

1. 탐구의 필요성

오목은 규칙이 단순하지만 경우의 수가 방대하여, 효율적인 게임 트리 탐색 알고리즘을 테스트하기 좋은 영역 중 하나이다. 기존의 미니맥스(Minimax)나 알파-베타 가지치기(Alpha-Beta Pruning) 같은 일반적인 탐색은 모든 경우의 수를 확인하므로 연산량이 너무 많다는 한계가 있다. 오목은 연속적인 공격(위협)을 통해 승패가 결정되는 게임이다. 따라서 일반 탐색 대신 승패와 직결되는 수만 골라 집중적으로 탐색하는 위협공간탐색 알고리즘을 탐구하여 그 개념과 효율성을 확인할 필요가 있다.

2. 탐구의 목적

교과서 속 본문 ‘Artificial Intelligence and the Arts’에서는 인공지능 프로그램이 렘브란트를 모방한 과정 및 인공지능의 학습 패턴과 인간 화가의 창작 과정의 유사성을 다룬다. 이처럼 《Go-Moku and Threat-Space Search》를 읽으며 인공지능 Victoria가 인간의 직관과 수읽기를 모방하여 방대한 선택지를 획기적으로 줄인 과정을 이해하고자 한다. 또한 이러한 과정에서 글의 전후 맥락과 논리적 구조를 바탕으로 낯선 단어의 의미를 스스로 추론하는 독해 역량을 함양하는 데 목적이 있다.

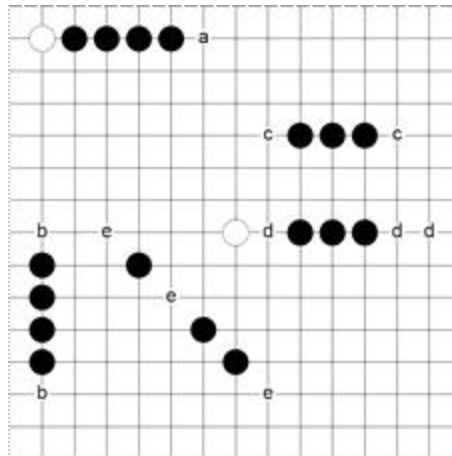
II. 탐구 내용

1. 위협공간탐색의 개념과 원리

가) 용어의 정의

1) 오목의 용어 정의

오목에서는 돌이 놓인 모양에 따라 상태를 정의하는 용어가 다르다. 사(four, 그림 1a)는 다섯 칸의 직선 중 공격수가 네 칸을 차지하고 다섯 번째 칸은 비어 있는 상태로 정의된다. 이와 달리 열린 사(straight four, 그림 1b)는 여섯 칸짜리 직선에서 공격수가 가운데 네 칸을 차지하고 양 끝의 두 칸은 비어 있는 상태를 말한다. 또한 삼(three, 그림 1c와 그림 1d)은 공격수가 가운데 세 칸을 차지하고 나머지 네 칸은 비어 있는 일곱 칸짜리 직선이거나, 가운데 네 칸 중 연속된 세 칸을 공격수가 차지하고 나머지 세 칸은 비어 있는 여섯 칸짜리 직선을 의미한다. 마지막으로 뿔 삼(broken three, 그림 1e)은 가운데 네 칸 중 연속되지 않은 세 칸을 공격수가 차지하고, 나머지 세 칸은 비어 있는 여섯 칸짜리 직선을 말한다.



[그림1. 위협]

2) 위협공간탐색의 용어 정의

위협 공간 탐색에서는 다양한 용어가 사용된다. 우선 이득 칸(gain square)은 공격수가 돌을 놓은 위치를 말하고 비용 칸(cost square)은 해당 위협에 대응하여 방어수가 돌을 놓은 칸들을 말한다. 이때 나머지 칸(rest square)은 위협이 성립할 가능성을 포함하고 있는 칸 중에서 이득 칸을 제외한 칸들을 뜻한다. 그리고 위협 A의 나머지 칸 중 하나가 위협 B의 이득 칸인 경우, 즉 위협 B의 이득 칸으로 인해 새로운 위협 A가 생기는 경우 위협 A는 위협 B에 의존한다고 한다.

나) 위협공간탐색의 원리와 예시

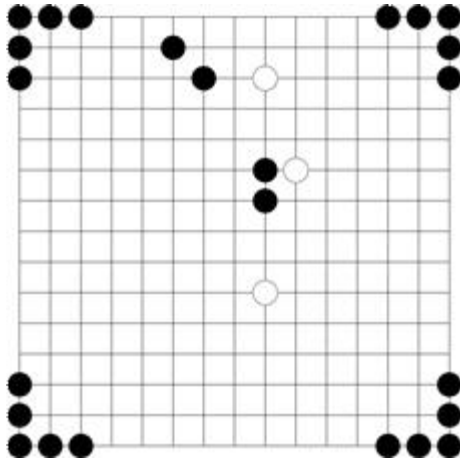
1) 위협공간탐색의 원리

위협공간탐색은 현재 공간에서 만들 수 있는 모든 독립적인 위협에 대해 트리를 만들며 시작한다. 이후 각 위협의 이득 칸으로 인해 새로운 위협이 생길 수 있는지 확인 한 뒤, 있다면 그 위협을 자식 노드로 추가한다. 트리를 확장하다가 어떤 가지가 열린 사나 오목으로 끝나면 이는 잠재적 승리 수순을 발견했다는 것을 의미한다. 기존의 알고리즘은 백의 방어수에 따라 트리가 나뉘지고 가지 수가 기하급수적으로 늘어나게 되지만 위협공간탐색의 트리 확장 과정은 백이 모든 비용 칸을 방어한 것으로 간주하고 다음 위협을 계속 이어나가 매우 효율적이다. 하지만 백의 공격을 고려하지 않았으므로 잠재적 승리 수순이 확정된 승리 수순이 아니므로 추가적인 검증이 필요하다.

2) 위협공간탐색의 예시

그림2와 표1에 이러한 위협공간탐색의 작동 방식이 잘 드러나 있다. 이 중 알고리즘이 찾아낸 승리 수순만 살펴보면, 첫 단계의 깊이 1에서 이득 칸이 e15고 비용 칸은 d15이다. 이는 흑이 e15에 두게 되면 가로 방향의 사가 생기게 되어, 백이 d15에 두어 이를 막는다는 걸 의미한다. 이후 깊이 2에서 흑이 i8에 두며 대각선 방

향으로 열린 사가 생기며 백은 i7을 막을 수 있지만 세로 방향으로 만들어진 삼은 이전 위협에 의존적이지 않아 i7, i11, i12를 막지 못하고 이로써 흑의 승리가 확인된다.



[그림2. 실제 위협이 있는 위치]

Depth	Type of threat	Gain square	Cost squares
1	Four	i15	k15
1	Four	k15	i15
1	Four	e15	d15
2	Four	i11	k12
3	Straight Four	i5	i7
2	Four	k12	i11
1	Four	d15	e15
1	Four	o12	o11
1	Four	o11	o12
1	Four	a12	a11
1	Four	a11	a12
1	Three	i11	i7,i8,i12
2	Four	k12	e15
2	Four	e15	k12
3	Five	d15	
1	Three	i5	i7,i11,i12
1	Four	o5	o4
1	Four	o4	o5
1	Four	i1	k1
1	Four	k1	i1
1	Four	e1	d1
1	Four	d1	e1
1	Four	a5	a4
1	Four	a4	a5

[표1. 그림1의 탐색 트리]

2. 위협공간탐색의 한계와 보완

Polygon이라는 프로그램이 앞서 연구한 11개의 국면을 해당 알고리즘의 효율성을 테스트하는 데 사용한 결과, 8개의 국면은 기존의 프로그램보다 더 효율적인 승리 수순을 발견했지만 3개의 국면에 대해서는 승리 수순을 발견하지 못했다. 이는 위협공간탐색이 트리 탐색 시 비용 칸을 모두 백이 막는다고 가정해 탐색 공간이 축소되어 나타나는 현상으로 Victoria에서는 위협공간탐색이 실패하면, 증명수 탐색 (Proof-Number Search) 알고리즘을 추가로 실행하는 방식으로 오목에서 흑의 승리를 증명하였다.

III. 결론

1. 요약

본 탐구에서는 오목에 적용되는 위협공간탐색 알고리즘의 개념과 원리를 탐구하였다. 먼저 오목에서 사용되는 용어(사, 열린 사, 삼, 뿔 삼)와 위협공간탐색에서 사용되는 용어(이득 칸, 비용 칸, 나머지 칸, 위협 간의 의존 관계)의 정의를 정리하였다. 이를 바탕으로 위협공간탐색이 독립적인 위협들을 중심으로 탐색 트리를 구성하고, 백이 모든 비용 칸을 방어한다고 가정함으로써 기존의 미니맥스나 알파-베타 가지치기보다 훨씬 효율적으로 승리 수순을 찾아내는 원리를 실제 예시와 함께 확인하였다. 나아가 Polygon 프로그램을 이용한 검증 결과 8개의 국면에서는 효율적인 승리 수순을 발견했지만 3개의 국면에서는 실패했다는 사실을 통해 위협공간탐색의 한계를 확인하였고, Victoria가 이를 증명수 탐색과 결합하여 보완한 사례를 통해 알고

리즘의 실용적 활용 방안까지 살펴보았다.

2. 느낀 점

이번 탐구를 진행하며 원문 논문인 《Go-Moku and Threat-Space Search》를 다섯 번 넘게 반복해서 읽었다. 사전에서 찾아보아도 정확한 의미를 파악하기 어려운 용어들이 많아 처음에는 문장 하나를 이해하는 데도 오랜 시간이 걸렸지만, 문단의 앞뒤 맥락과 논리적 흐름, 그리고 그림과 표에 제시된 구체적인 예시를 함께 살펴보며 낯선 단어의 의미를 스스로 추론해 나갔다. 이러한 과정을 여러 차례 반복하면서 단어의 사전적 뜻에만 의존하기보다 글 전체의 구조와 흐름 속에서 의미를 파악하는 문맥 추론 능력이 크게 향상되었음을 느낄 수 있었다. 또한 복잡한 영어 원서를 스스로 해석해 나가는 경험을 통해 독해 실력 또한 눈에 띄게 성장했다고 생각한다. 이번 탐구는 인공지능 알고리즘에 대한 이해를 넓히는 데 그치지 않고, 낯선 영어 지문을 능동적으로 읽어내는 힘을 기르는 뜻깊은 경험이 되었다.

[참고자료 기재]

<참고 문헌>

- L.V. Allis, H.J. van den Herik, & M.P.H. Huntjens, 「Go-Moku and Threat-Space Search」, 『Computational Intelligence』, 1994.
- <https://www.baeldung.com/cs/gomoku-threat-space-search>